

Self-locking Multi-Diameter Cylindrical Column Climber

Ruizhi Deng*, Hengyi Zhou*, Dachan Liu*, Letao Zhang*, *Shanghai Jiao Tong University*

Abstract—This report introduces a column-climbing robot that can help inspect and repair the damaged overpass bridge piers, which are commonly seen in daily life due to fatigue and abrasion. To achieve the objective, the robot needs to climb up and down steadily and self-locking is also required to enable inspection and repairing work. Our robot features a stiff frame which helps enhance the climbing stability, a specially designed clamp that can ensure self-locking, and a laser sensor to ensure the locking height. Final test shows that the robot can climb up, stop at the 120 cm height, and move down to the ground on two columns of different diameter within 5 minutes, without any slipping. In conclusion, our robot shows great performance and has a promising application in terms of bridge pier repairing.

Index Terms—Article submission, IEEE, IEEEtran, journal, L^AT_EX, paper, template, typesetting.

I. INTRODUCTION

HIGHWAY overpasses play a vital role in our daily commuting and transportation, significantly promoting landscape connectivity [1]. However, the bridge piers are often faced with several technical challenges, such as corrosion, load-induced fatigue and abrasion damage. Under this circumstance, manual inspections and repairing can be dangerous. Therefore, a vertical climbing robot, appears to be the most suitable solution. Additionally, to enable repairing work, the robot also needs to achieve self-locking at any given height.

A quite deal of models with these functions have been developed to fulfill the pier repairing task. Although the goal is achieved by different means, the models can be simply divided into 3 main categories. The first category features grippers that work in cycle to grasp the column while the body extends or compresses during a full gait cycle [2]. 3DCLIMBER from University of Coimbra [3], Climbot from South China University of Technology [4], Treebot from Shenzhen Institute of Advanced Technology [5], CCRobot from The Chinese University of Hong Kong [6] are all typical examples of models applying the gripping mechanism. However, the disadvantages of this kind of approach is quite obvious, namely: slow climbing speed, poor precision in vertical location control (hard to reach the distance of non-integer numbers of working cycles). Designs from the second category connect several robots with wheels together to achieve the objective, such as the Wheel Type Tree-Climbing Robot from Zhejiang University [7] and SMART from University of Applied Sciences Aachen [8]. Although these designs are relatively easy to manufacture, they exhibit poor stability during the climbing process due to the speed difference between parts. The last

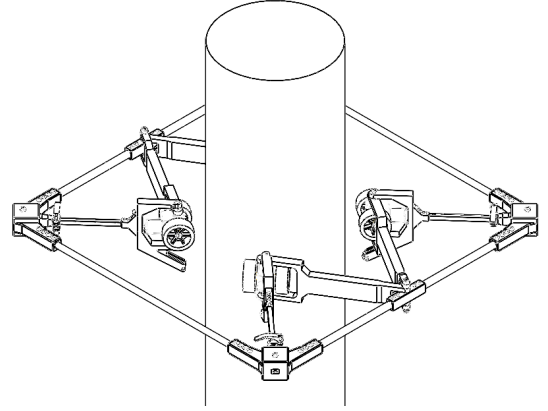


Fig. 1. Overall design.

category features a whole enclosing frame with wheel attached on the frame, like the wind turbine climbing robot from Southern Taiwan University of Science and Technology [9], the human-inspired pole climbing robot from University of Tehran [10], the wheeled pole-climbing-robot [11] and the tree pruning robot from Massey University [12]. These designs provide high climbing speed and stability during climbing process, but they encounter problems when it comes to locking mechanism design and columns with varying diameter. (For more information about literature review, please refer to the Literature Review Table (Table VII) in the Appendix for reference.)

We base our design on the models of the third category because of the high working speed and stability. Similarly, we build the frame by connecting 2 aluminium (Al) alloy tubes and 2 carbon fiber reinforced plastics (CFRP) tubes with 3-D printed connectors. To deal with the varying diameter problem, we fixed the wheels on a polycarbonate (PC) sliding link that can steplessly slide relative to the frame so that our robot fits columns of almost all sizes. Moreover, for self-locking mechanism, special clamp mechanism are designed (which will be introduced in detail in Design part) to fix the sliding link when needed. Additionally, the plastic deformation of PC link and the rubber wheels can provide sufficient normal force on the column, and consequently, the friction is large enough to lock the whole robot. During the test, our design proved capable of climbing up the column with desirable speed and stability. Furthermore, due to our improvements on the third method, our design fits columns of almost any diameter and can achieve self-locking even without power supply.

Fig. 1 is the overall design and Fig. 2 is the real photo.

*These authors contributed equally to this work.



Fig. 2. Real photo.

II. DESIGN

According to the game rule, the robot must be capable of climbing two vertical columns with different diameters: 300 mm and 600 mm. To meet this requirement, we adopted a four-bar linkage mechanism with a single sliding joint. A two-position synthesis method was applied to determine the linkage geometry that fits for both diameters. Meanwhile, to ensure stable climbing and prevent lateral deviation, the four driving modules should be evenly distributed around the column, with module centered along the bisector of one quadrant of the column's circumference. Additionally, the transmission angle of the linkage must remain sufficiently large to generate adequate radial force – especially for the column with 300 mm diameter.

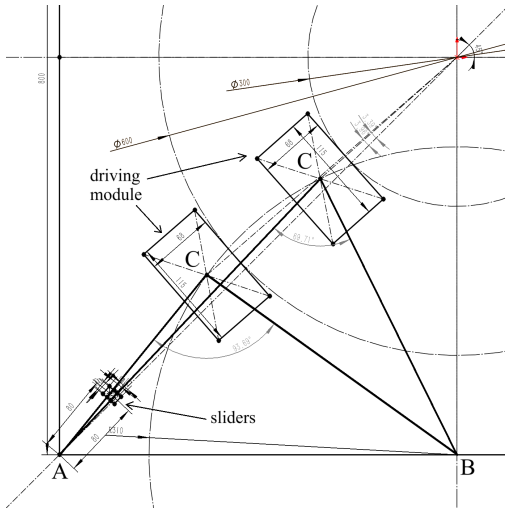


Fig. 3. Link design of 1/4 of the robot.

Therefore, we carefully synthesized and optimized the dimensions of the mechanism, with results illustrated in Fig. 3 and Fig. 4. Joint A and C are grounded to the robot frame. B connects the driving module with the linkage. The sliding joint is installed between joint A and B, and the linkage length are set as follows:

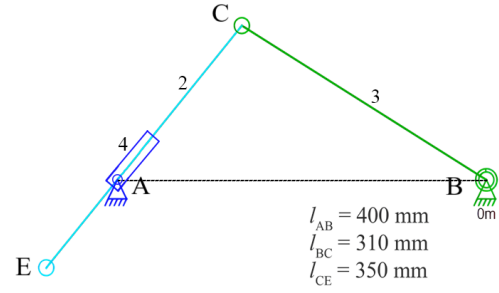


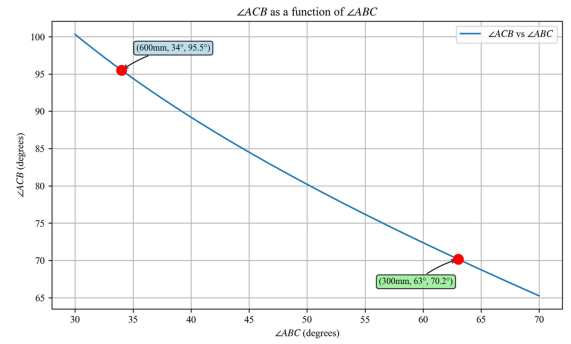
Fig. 4. Simplified link diagram.

$$l_{AB} = 400 \text{ mm}$$

$$l_{BC} = 310 \text{ mm}$$

$$l_{A\text{-sliderstart}} = 80 \text{ mm}$$

This schematic clearly demonstrates that our linkage design can effectively press the driving module against the column with an efficient transmission angle from 69.71° to 93.69° without any mechanical interference, as shown in Fig 5.

Fig. 5. Transmission angle $\angle ACB$ w.r.t. $\angle ABC$.

Compared with existing solutions, our design poses several advantages. First, our robot can self-lock on the column without power supply for a long time. This contrasts with designs using suction units [13], which require continuous power to maintain vacuum pressure. Second, our method does not damage column surface. Previous designs that are able to maintain its position on the column utilizes some sorts of hooks, such as the design by Provancher et al. [14], which relies on dactyl claws for anchoring, may leave small indentations on the column surface. This might be unacceptable in certain applications. Our design uses wheels with high friction, which doesn't have such concern.

Several design choices contribute to these advantages. As previously discussed, the four-bar linkage effectively transfers radial force to the wheels in contact with the column. These wheels are made of a high-friction material, specifically selected to maximize grip, which will be discussed in the next section. Additionally, the motors used are worm-gear-based self-locking motors, which resist external back-driving. Together, these elements enable the robot to self-lock securely

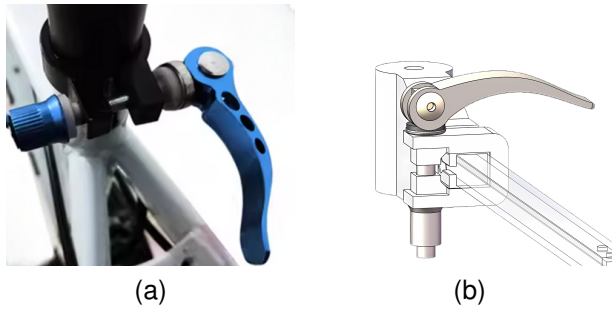


Fig. 6. (a) Bicycle-style seat post quick-release clamps. (b) How the Clamp is used on our robot.

onto the column without consuming power or damaging the surface.

One additional feature that enhances the portability of our design is the use of bicycle-style seat post quick-release clamps to lock the sliding link in place. These clamps significantly improve portability and ease of use compared to other attachment methods. Users can quickly loosen or tighten the clamps by simply toggling a lever, enabling rapid adjustments when switching between columns of different diameters. The process involves releasing the clamps, repositioning the linkages, and securing them again—requiring no tools. This mechanism is shown in Fig. 6.

The final design of our robot is illustrated in Fig 7.

III. MANUFACTURING

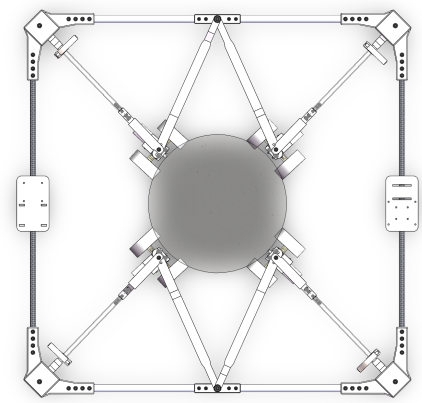
In this section, the fabrication and assembly of our robot are discussed.

A. Material Selection

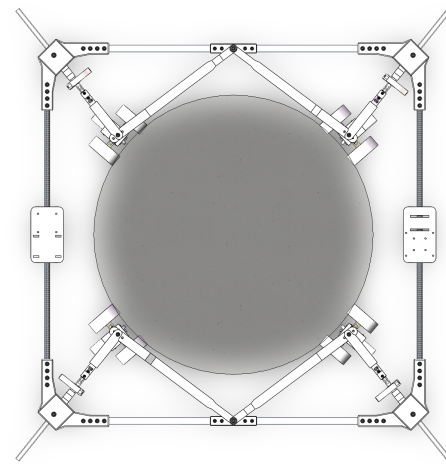
1) *Frame Materials*: The material used for the robot frame should be stiff and light-weighted. According to the Ashby Chart (Fig. 8), Al alloys and carbon fiber (CFRP) are two great choice for our design, since they are relatively have high Young's modulus yet small density. This ensures our frame to be stiff enough to resist large force with small deformation, and our robot to be light enough.

2) *Sliding Link Material*: The sliding link (Link CE in Fig.4) is made from polycarbonate (PC) due to its characteristic of relatively low Young's modulus and low density, as illustrated in Fig.8. These material properties align well with the functional requirements of this component: it must bend a little bit to generate a compliant pressing force that maintains tight contact between the driving module and the column surface. At the same time, the material must withstand repeated deflection without fracturing. As a result PC can accommodate due to its favorable toughness and elasticity.

3) *Connecting Part Materials*: The connecting parts of our robot, such as the frame corner connector and the motor case, are made of PLA using 3D printing. We choose PLA because it has relatively high material strength, it is light-weighted, and more importantly, it is cheap and easy to print using 3D printers.



(a)



(b)

Fig. 7. Top view of our final design climbing (a) 300mm column (b) 600mm column.

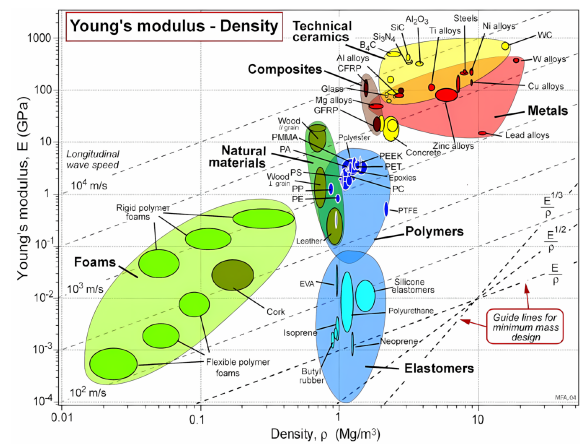


Fig. 8. The Ashby chart for material selection [15].

4) *Motor Selection*: We choose the motor worm-gear JGY-370 (12V, 30RPM) to be our motor (Fig. 9). Details about this motor is listed in Table I. It can be seen that this motor is suitable for our robot because (a). it can resist external torque



Fig. 9. JGY-370 motor.

on the output shaft, thanks to its worm gear structure and (b). it provides large torque for the robot to climb the column (details of this will be provided in the analysis section).

TABLE I
MOTOR SPECIFICATIONS

Parameter	Value
Gear Ratio	200
No-Load Current	≤ 60 mA
No-Load Speed	30 rpm
Rated Voltage	12 V
Rated Torque	5.3 kg-cm
Rated Speed	22 rpm
Rated Current	≤ 0.6 A
Max Torque	10 kg-cm
Stall Current	1.7 A

5) *Sensors*: When selecting the distance measurement sensor, three candidates are evaluated: the HC-SR04 ultrasonic sensor, the TOF200F laser range sensor, and the YB-MVV18 VER1.0 (VL53L0X-based) TOF sensor. The comparison criteria included accuracy, range stability, data consistency, communication interface, and environmental robustness. Table IX summarizes the key characteristics of each sensor.

Although the HC-SR04 is easy to use and cheap, it suffers from poor accuracy and stability as the measured distance increases, especially when the measured distance exceeds 1 m. The TOF200F has erratic outputs such as unrealistic readings, hence it is not reliable enough. In contrast, the YB-MVV18 VER1.0 provides stable and accurate results, with specifications satisfying our needs (Table. II), so it is selected to be the sensor to measure the height of our robot on the column.

TABLE II
SPECIFICATIONS OF YB-MVV18 VER1.0 SENSOR

Parameter	Value
Sensor Model	VL53L0X
Measurement Range	1.5 cm – 200 cm
Measurement Accuracy	± 3
Measurement Time	20 ms (max)
Operating Voltage	DC 3.3 V / 5 V
Operating Current	12–20 mA
Communication Interface	I ² C
I ² C Address	0x29
Communication Frequency	400 kHz (max)
Laser Wavelength	940 nm
Field of View	25°
Operating Temperature	–20°C to 70°C
Module Size	29.4 mm × 36 mm

B. Manufacturing Procedure

1) *3D Printing*: Most parts of our robot, including the frame corner connector, the beam connecting the driving module and the frame corner, the motor case and other parts of irregular shapes are 3D printed using PLA at 40% fill to provide enough strength.

2) *Sawing and Drilling*: The PC beams, aluminum beams and carbon fiber beams are sawed to the designated length. Holes on them are manually drilled.

C. Assembly Procedure

1) *Driving Module Assembly*: Use screws and nuts to install the JGY-370 motor together with its support inside the driving module support.

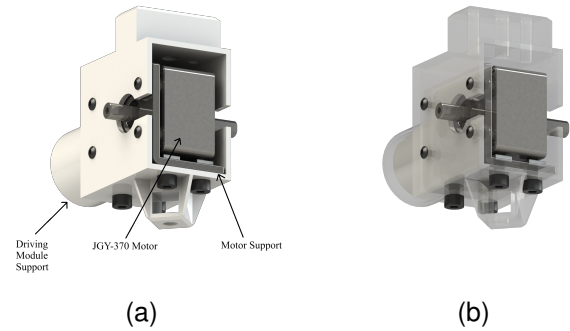


Fig. 10. Install the motor into the driving module support. (a) real view (b) transparent view.

Then assemble two silicone rubber wheels onto the output shaft using couplings and screws.

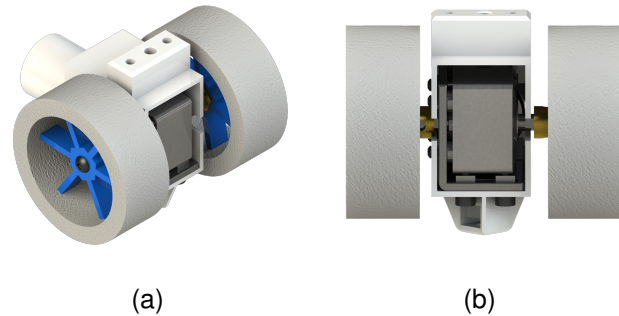


Fig. 11. Assemble the wheels onto the motor (a) isometric view (b) right view.

After that, assemble the PC tube and the driving module inner clamp together. Then install the auxiliary wheel.

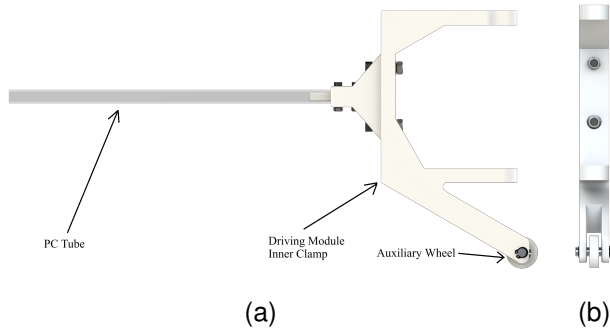


Fig. 12. Assemble the PC tube and the inner clamp (a) side view (b) front view.

Then install the this component and the driving module outer - central base inner/outer connector to the driving module.

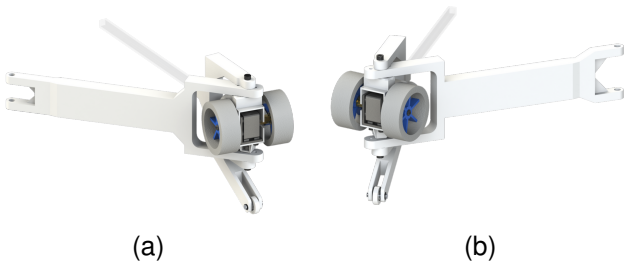


Fig. 13. Completed (a) driving module (inner) (b) driving module (outer).

At this point, the assembly of the drive module is complete. Each inner/outer drive module requires two units.

2) *Frame Assembly*: Install the central base onto the two aluminum tubes using screws and nuts.

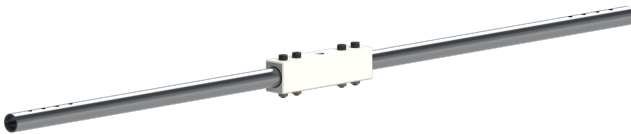


Fig. 14. Install the central base.

Put the frame corner connector PC clamp, frame corner connector and the quick-release clamp together, and install them onto the frame corner base.

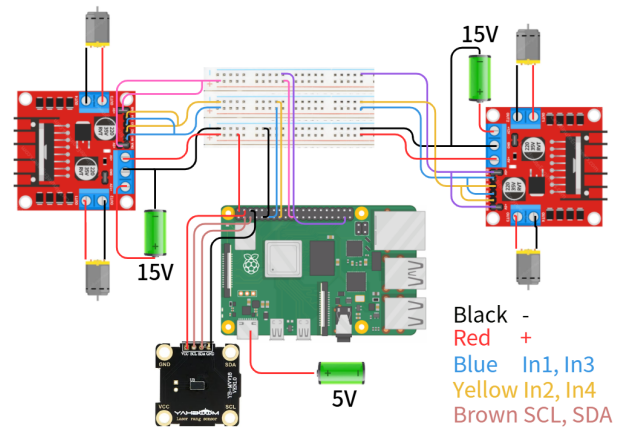


Fig. 17. Circuit diagram.

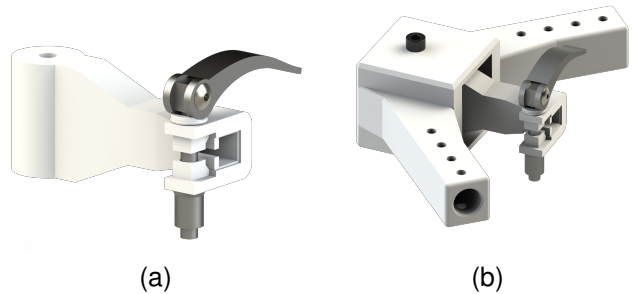


Fig. 15. Completed (a) corner connector with quick-release clamps (b) corner component.

Assemble 2 carbon fiber tubes with casings, 2 aluminum tubes, and 4 corner components assembled in the previous step together to finish the assembly of the robot frame.

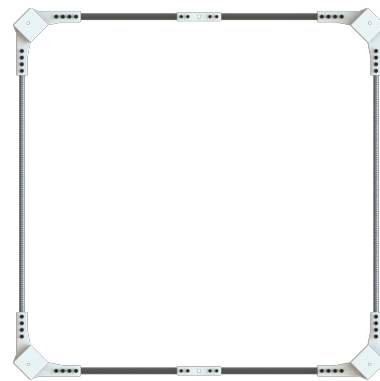


Fig. 16. Completed robot frame.

3) *Robot Assembly*: Install the 4 driving modules (2 outer 2 inner) onto the frame, pass the PC tube through the corner connector.

4) *Sensor Assembly and Wiring*: Assemble the Raspberry Pi, the L298N motor controller, the YB-MVV18 distance sensor, the control button and the batteries onto the robot frame. Then wire these components according to Fig. 17.

Now the assembly of our robot is finished. The complete robot is shown in Fig. 18.

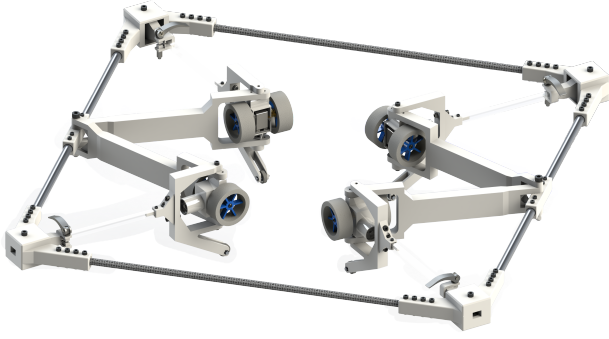


Fig. 18. Complete view of the robot.

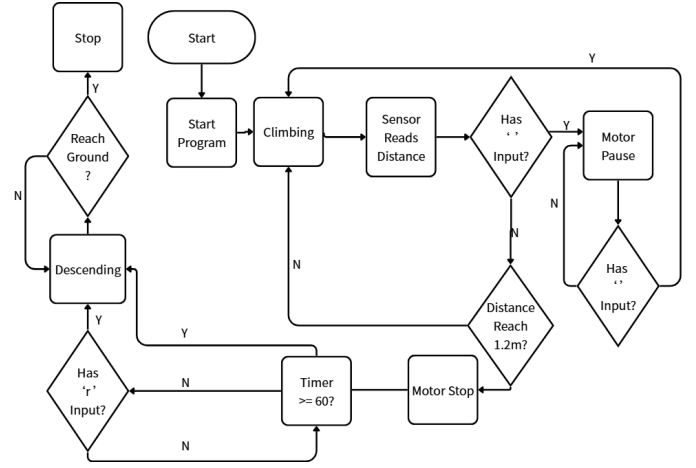


Fig. 19. Logic diagram.

IV. CONTROL OF SENSORS AND ACTUATORS

A. Control Circuit

In this project, we use Raspberry Pi as the central digital element to control the circuits. Two L298N is connected to the corresponding pins on the Raspberry Pi. For each of the L298N, two motors are connected to it. All of the motors are adjusted to have the same direction at the same time, but the rotational speeds are controlled respectively for each pair of motors. Two batteries with a voltage of 15V are provided for each pair of motors, and a battery with a voltage of 5V is provided for the logic of Raspberry Pi. Raspberry Pi provides 5V for the two L298N for its logic circuit. We use a distance laser sensor whose type is YB-MVV18 VER1.0 to detect the distance between it and the ground to determine the motion of the movement. Raspberry Pi provides 5V for the sensor to work. The circuit diagram is shown in Fig. 17

B. Logic Diagram

The logic diagram of the implementation is shown in Fig. 19. At the beginning, we start the program, then the motor moves forward and the robot climbs the cylinder. Meanwhile, the laser sensor starts to detect the distance between the robot and the ground. The system will then detect whether there is an input of the empty space on the keyboard of our computer. If there is, the motor stops, and only when the empty space is tapped again does the motor move forward again. If there is no input of empty space, the robot will not stop until it reaches a height of 1.2m. Then the system checks whether 60 seconds have passed from the time when the motor stopped. If it has, the robot starts descending. Otherwise, the system checks whether there is an 'r' input from the keyboard. If there is, the robot descends immediately without considering the timer. If there is not, the system continues to count the timer. Finally, the robot does not stop until it reaches the ground. We choose the top of the robot to be the reference point, and the sensor has a vertical distance of 20cm with respect to the top point. Therefore, the robot will stop at around distance of around 100cm above the ground.

C. Sensor Analysis

TABLE III
LASER SENSOR DISTANCE VALUES

Index	Distance in cm	Distance in mm
1	120 cm	1203 mm
2	120 cm	1207 mm
3	120 cm	1200 mm
4	121 cm	1211 mm
5	121 cm	1210 mm
6	120 cm	1208 mm
7	120 cm	1209 mm
8	120 cm	1205 mm
9	121 cm	1210 mm
10	120 cm	1209 mm
11	121 cm	1214 mm
12	120 cm	1206 mm
13	120 cm	1205 mm
14	121 cm	1215 mm
15	121 cm	1212 mm
16	122 cm	1220 mm
17	121 cm	1213 mm
18	120 cm	1207 mm
19	120 cm	1203 mm
20	122 cm	1222 mm
21	121 cm	1219 mm
22	122 cm	1226 mm
23	122 cm	1220 mm
24	122 cm	1221 mm
25	121 cm	1218 mm
26	122 cm	1220 mm

We have tested several kinds of sensors shown in Table IX to evaluate their performance. As discussed previously, the HC-SR04 Ultrasonic sensor and TOF200F laser sensor both have their own problems. The YB-MVV18 VER1.0 laser distance sensor performs pretty well. There are a total of 26 distance values shown in Table III as the robot reaches a height of 1.2m. The average distance is calculated by:

$$\frac{\sum_{i=0}^{26} d_i}{26} = \frac{31513}{26} = 1212.04\text{mm} \quad (1)$$

Where d_i represents every single value of distance. Then the average error percentage is:

$$\frac{1212.04 - 1200}{1200} \times 100\% = 1.003\% \quad (2)$$

The maximum distance shown in the figure is 1226mm, thus the maximum error percentage is:

$$\frac{1226 - 1200}{1200} \times 100\% = 2.167\% \quad (3)$$

Therefore, the sensor works pretty well and meets the requirements of our project.

V. ANALYSIS

A. Linkage Classification

The linkage diagram of our design is shown in Fig. 4. Our linkage has 4 links with 3 rotating full joints and 1 sliding full joint. The DOF of this linkage can be calculated as

$$M = 3(L - 1) - 2J_1 = 3 \times (4 - 1) - 2 \times 4 = 1 \quad (4)$$

According to Fig. 4, our linkage is a crank-slider, which doesn't have a Grashof condition.

B. Position Analysis

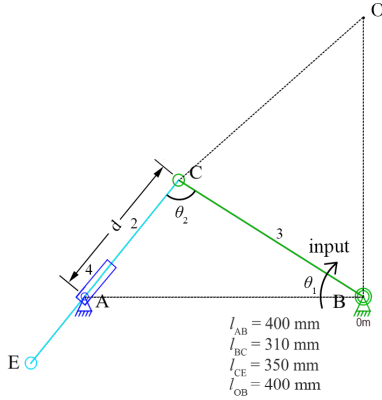


Fig. 20. Simplified link diagram with annotations.

According to Fig. 20, the unknown variables are θ_2 , d and l_{OC} . As θ_1 varies, these values can be expressed as

$$d = \sqrt{l_{AB}^2 + l_{BC}^2 - 2l_{AB}l_{BC}\cos\theta_1} \quad (5)$$

$$\theta_2 = \cos^{-1} \frac{d^2 + l_{BC}^2 - l_{AB}^2}{2dl_{BC}} \quad (6)$$

$$l_{OC} = \sqrt{l_{OB}^2 + l_{BC}^2 - 2l_{OB}l_{BC}\cos(90^\circ - \theta_1)} \quad (7)$$

where all angles are expressed in degrees.

How these variables vary with the change of θ_1 are shown in Fig. 21. Especially, when the robot is climbing the 300 mm column, i.e.

$$l_{OC} = r_{300} + r_{wheel} = 150 + 34 = 184 \text{ mm},$$

we have

$$\theta_1 = 63.70^\circ$$

$$\theta_2 = 69.68^\circ$$

$$d = 382.39 \text{ mm}$$

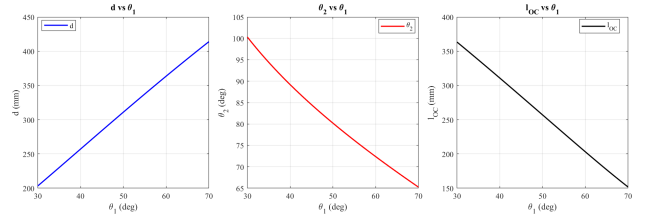


Fig. 21. Position Analysis.

When the robot is climbing the 600 mm column, i.e.

$$l_{OC} = r_{600} + r_{wheel} = 300 + 34 = 334 \text{ mm},$$

we have

$$\theta_1 = 35.70^\circ$$

$$\theta_2 = 93.64^\circ$$

$$d = 233.89 \text{ mm}$$

C. Force analysis.

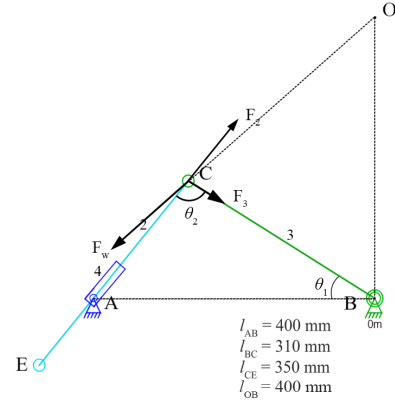


Fig. 22. Force analysis.

To simplify the problem, we abstract our system as the linkage system shown in Fig. 22:

- link AC: square PC tube with an out side length of 10 mm and a wall thickness of 1 mm
- link BC: PLA tube with a section dimension of 45.3 mm $\times$ 20 mm
- link AB: hollow aluminum tube with an out diameter of 15 mm and a inner diameter of 12.75 mm

In reality, we observe an approximately 1 mm deflection of PC tube. Below we show that only this deflection need to be considered in calculation.

Since the link AC (PC tube) undergoes the first-order buckling mode, with maximum bending b in the middle. Then, the deformation of the beam's midline can be approximated as

$$w(x) = b \sin\left(\frac{\pi x}{L}\right) \quad (8)$$

Plug into the differential equation for beam buckling:

$$EI \frac{d^4 w(x)}{dx^4} + F \frac{d^2 w(x)}{dx^2} = 0 \quad (9)$$

we obtain:

$$F_{cr} = F = \frac{\pi^2 EI}{L^2} \quad (10)$$

1) *600 mm Case:* First we consider the 600 mm case, when $L = 233.89$ mm. Plug in E and I , $F_{cr} = 213.04$ N, with $E_{PC} = 2.4$ GPa.

Assume that $F_2 = F_{cr}$. Since at joint C, we have:

$$\sum \vec{F} = \vec{F}_w + \vec{F}_2 + \vec{F}_3 = \vec{0} \quad (11)$$

from which we can get

$$\frac{F_w}{\sin \angle F_2 C F_3} = \frac{F_2}{\sin \theta_2} = \frac{F_3}{\sin \angle F_w C F_2}. \quad (12)$$

Thus $F_w = 218.44$ N, $F_3 = 36.84$ N.

The axial deformation of link BC and AC is given by

$$\begin{aligned} \Delta l_{BC} &= \frac{F_3 L}{AE} = 3.6 \mu\text{m} \ll 1 \text{ mm} \\ \Delta l_{AC} &= \frac{F_2 L}{AE} = 576.7 \mu\text{m} < 1 \text{ mm} \end{aligned}$$

so the deformation of link BC as well as link AC can be ignored.

F_w is the normal force the wheels of one driving module applied onto the column. The friction between the silicone rubber wheels and the column surface is static friction, since no slipping happens. Also, we know the friction coefficient of the silicone rubber wheel μ is approximately 0.5. Using these numbers, we can calculate the maximum friction that the 4 driving modules combined can provide:

$$f_{total} = 4\mu N = 4\mu F_w = 436.88 \text{ N} \quad (13)$$

In reality, the maximum friction is reached when the robot is accelerating when climbing up the column. The maximum acceleration is calculated using the data obtained from the distance sensor as

$$a = \frac{v_{max}}{t} = \frac{8.545 \text{ cm/s}}{100 \text{ ms}} = 0.8545 \text{ m} \cdot \text{s}^{-2} \quad (14)$$

The mass of this robot is 4.735 kg. Take $g = 9.8 \text{ m} \cdot \text{s}^{-2}$, and use the Newton's second law, we can obtain the maximum friction in reality:

$$f_m - mg = ma \quad (15)$$

where we get $f_m = 50.45$ N.

As a result, the safety factor of our robot when climbing the 600 mm column is given by

$$f_s = \frac{f_{total}}{f_m} = 8.66. \quad (16)$$

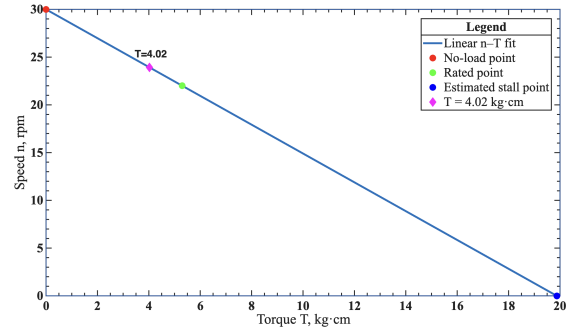


Fig. 23. n-T for the motor.

2) *300 mm Case:* Then we consider the 300 mm case, when $L = 382.39$ mm. Similarly, plug in E and I , $F_{cr} = 79.70$ N, with $E_{PC} = 2.4$ GPa.

Assume that $F_2 = F_{cr}$. Since at joint C, using equation (12) we can get $F_w = 77.50$ N, $F_3 = 7.20$ N.

The axial deformation of link BC and AC is given by

$$\begin{aligned} \Delta l_{BC} &= \frac{F_3 L}{AE} = 0.704 \mu\text{m} \ll 1 \text{ mm} \\ \Delta l_{AC} &= \frac{F_2 L}{AE} = 412 \mu\text{m} < 1 \text{ mm} \end{aligned}$$

so the deformation of link BC as well as link AC can also be ignored.

Using the same steps as described in the 600 mm case,

$$f_{total} = 4\mu N = 4\mu F_w = 155 \text{ N} \quad (17)$$

with the real maximum friction remaining the same: $f_m = 50.45$ N.

As a result, the safety factor of our robot when climbing the 300 mm column is given by

$$f_s = \frac{f_{total}}{f_m} = 3.07. \quad (18)$$

D. Climbing Speed Analysis

Though for 300 mm and 600 mm pier, the robot has different racial force, the friction (driving force) is the same. The friction has nothing to do with the normal force or racial force since the wheels rotate purely on the surface of the piers which causes static friction.

See table I for motor details. The performance of the motor is displayed in Fig. 23.

Since the whole weight of the robot is 4.735 kg, we have

$$T_D = m \times r_w = 4.735 \text{ kg} \times 3.4 \text{ cm} = 16.099 \text{ kg} \cdot \text{cm}. \quad (19)$$

which means $T = \frac{1}{4} T_D = 4.02 \text{ kg} \cdot \text{cm}$. Based on the n-T plot, this gives the $n = 23.97 \text{ rpm}$. So, the climbing speed should be

$$v = n \cdot c = 0.4 \times \pi \times d = 8.545 \text{ cm/s}. \quad (20)$$

VI. EXPERIMENT

A. Rolling Speed

The highest point of our robot is 20 cm away from the ground. So, when the highest point arrives 120 cm, the whole robot climbs about 100 cm.

For both 300 mm diameter and 600 mm diameter, we do the speed test for 5 times for up and down and take the average time needed and then calculate the average speed. The results are as follows:

TABLE IV
SPEED EXPERIMENT RESULT

	300 mm	600 mm
Average Up Speed	6.25 cm/s	7.41 cm/s
Average Down Speed	9.74 cm/s	9.52 cm/s

In our theoretical calculation of climbing speed, all the four speed should be approximately 8.545 cm/s (See section Climbing Speed Analysis). However, the error table is a little bit disgusting:

TABLE V
SPEED EXPERIMENT RESULT ERROR

	300 mm	600 mm
Average Up Speed	36.72%	15.32%
Average Down Speed	12.27%	9.52 %

Though the error is very huge, it's easy to find the reason. Watching the test videos, we can tell that the movement is not perfect pure rotation. There must some sliding included. When the pier become smaller, the sliding become more significant due to the structure of our robot and the sliding is also more significant when downwards than upwards because of the gravity. This phenomenon actually shows the difference of friction forces or driving forces, which violates the assumption above. However, considering sliding is too complex for us since it is influenced by too many elements that difficult to quantify, including the irregularity of the surface of the piers and the calculation will be out of control. So we have to simplify the model and get a rough simulation.

B. Locking Capacity

We have done the calculation in Force Analysis part. The f_s is 8.66 and 3.07 respectively.

Then demonstration video has been uploaded to the Bilibili with url: https://www.bilibili.com/video/BV15tpzkEQn/?vd_source=d9b9d6ab2a320e2c549cedb199c6952c

VII. DISCUSSION

A. Analysis of Differences between Theoretical Calculations and Experimental Results

We calculate a theoretical climbing speed of 8.545 cm/s. However, experimental results show that the actual average upward speed is between 6.25 cm/s and 7.41 cm/s, and the average downward speed is between 9.52 cm/s and 9.74 cm/s, with an error margin of 9.52% to 36.72%. This discrepancy could be attributed to several factors. Firstly, the theoretical

calculation considered static friction while neglecting rolling friction between the wheels and the column surface, friction losses within the mechanical transmission system, such as gears, and efficiency losses in the worm gear motor during actual operation. These additional forces would reduce the actual climbing speed.

Secondly, although the frame of the robot uses the material of carbon tube and aluminum tube, the connector links between the driving module and the frame are still made of PC material. The theoretical model may not have fully accounted for the minor deformation of the PC sliding links under load, which would affect the actual normal force exerted by the driving modules on the column, thus changing the magnitude of friction and the resulting speed.

B. Strengths and Successes of the Design

Despite the speed discrepancies, the design achieved great success in satisfying its function.

Firstly, the robot can self-lock without a power source. This design advantage distinguishes it from suction-cup robots that require continuous power to maintain a vacuum, significantly enhancing operational safety and energy efficiency.

Secondly, the use of a four-bar linkage and a sliding joint, combined with a bicycle-style quick-release clamp, allows the robot to be quickly adjusted without tools to fit different column diameters, which improves its flexibility.

Thirdly, the material selection is pretty significant, using high-rigidity aluminum alloy and carbon fiber reinforced plastic for the frame materials, which ensures the robot does not deform under heavy loads. Similarly, selecting an elastic PC tube for the sliding links allows to maintain a tight contact between the driving modules and the column.

VIII. CONCLUSION

In this report, we introduce a column-climbing robot designed for pier damage inspection and repairing. The robot can climb up and down the bridge piers of highway overpasses to inspect whether the piers are damaged. Moreover, it can achieve self-locking at any given height so that it can also serve as a stable platform for installing repairing robot arms. Our robot mainly consists of a frame with high stiffness and 4 pairs of wheels fixed on sliding links extended from the corner of the frame. For self-locking mechanism, we get the inspiration from bicycle-style seat post quick-release clamps and use those clamps to immobilize the sliding link when needed.

In the whole process, through endless trial and error, we improve our design and promote our understanding of such mechanisms. Here are several vital findings during the process:

- think thoroughly: during CAD process, import all the parts, including the nuts and bolts. Otherwise, interference may occur and makes assembly work much harder.
- inspiration from daily life: we design our locking mechanism based on the observation of bicycle-style seat post quick-release clamps.
- make use of material properties: our frame is made of material with high stiffness, so it won't deform when

climbing, while sliding link is made of PC which is relatively elastic and can provide normal force on the wheels.

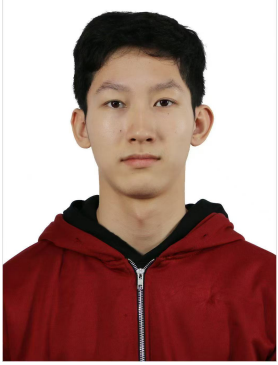
During the tests, our robot proved capable of accomplishing all the objectives for our design. It can climb steadily up and down two columns in less than 5 minutes. Moreover, during the self-locking phase, it can hold on to the column without slipping. In conclusion, our robot demonstrate great performance in column-climbing and self-locking, promising great potential in future applications, bridge pier damage inspection and repairing.

REFERENCES

- [1] M. P. Olsson, P. Widén, and J. L. Larkin, "Effectiveness of a highway overpass to promote landscape connectivity and movement of moose and roe deer in sweden," *Landscape and Urban Planning*, vol. 85, no. 2, pp. 133–139, 2008.
- [2] R. Poon and I. Hunter, "Advances in hybrid modular climbing robots: Design principles and refinement strategies," 2025. [Online]. Available: <https://arxiv.org/abs/2503.07423>
- [3] M. Tavakoli, A. Marjovi, L. Marques, and A. T. de Almeida, "3dclimber: A climbing robot for inspection of 3d human made structures," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2008, pp. 4130–4135.
- [4] Y. Guan, L. Jiang, H. Zhu, X. Zhou, C. Cai, W. Wu, Z. Li, H. Zhang, and X. Zhang, "Climbot: A modular bio-inspired biped climbing robot," in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2011, pp. 1473–1478.
- [5] T. L. Lam and Y. Xu, "A flexible tree climbing robot: Treebot - design and implementation," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 5849–5854.
- [6] Z. Zheng, S. Hu, and N. Ding, "A biologically inspired cable climbing robot: Ccrobot - design and implementation," in *2018 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2018, pp. 2354–2359.
- [7] D. Ren, S. Yang, G. Yan, and Y. Zhang, "Study on a novel wheel type tree-climbing robot," in *2014 Seventh International Symposium on Computational Intelligence and Design*, vol. 1, 2014, pp. 150–153.
- [8] F. Josef, D. Shengzhi, K. Stephan, D. Enno, and E. Heiko, "Design of a multi-robot system for wind turbine maintenance," *Energies*, vol. 13, no. 10, 2020.
- [9] L. Jui-Hung and P. Kathleen, "Design and development of a climbing robot for wind turbine maintenance," *Applied Sciences*, vol. 11, no. 5, 2021.
- [10] A. Sadeghi, H. Moradi, and M. N. Ahmadabadi, "Analysis, simulation, and implementation of a human-inspired pole climbing robot," *Robotica*, vol. 30, no. 2, p. 279–287, 2012.
- [11] W. FISCHER and R. SIEGWART, *WHEELED POLE-CLIMBING-ROBOT WITH HIGH PAYLOAD CAPABILITY, USING A CLAMPING MECHANISM WHICH IS INSPIRED BY THE ROPE-CLAMPS IN HUMAN CLIMBING*, pp. 399–406. [Online]. Available: https://www.worldscientific.com/doi/abs/10.1142/9789814329927_0050
- [12] P. Gui, L. Tang, and S. Mukhopadhyay, "A novel robotic tree climbing mechanism with anti-falling functionality for tree pruning," *Journal of Mechanisms and Robotics*, vol. 10, no. 1, p. 014502, 11 2017. [Online]. Available: <https://doi.org/10.1115/1.4038219>
- [13] K. Shi and X. Li, "Vacuum suction unit based on the zero pressure difference method," *Physics of Fluids*, vol. 32, no. 1, Jan. 2020.
- [14] W. R. Provancher, S. I. Jensen-Segal, and M. A. Fehlberg, "Rocr: An energy-efficient dynamic wall-climbing robot," *IEEE/ASME Transactions on Mechatronics*, vol. 16, no. 5, pp. 897–906, Oct. 2011.
- [15] P. Engineering. (2020) Material selection charts — right material for engineering design,. [Online]. Available: <https://proxom.net/material-selection/>
- [16] M. Nor Faizal, W. Othman, and S. Syed Hassan, "Development of pole-like tree climbing robot," in *2015 IEEE International Conference on Control System, Computing and Engineering (ICCSCE)*, 2015, pp. 224–229.
- [17] Y. Li, M. Z. Q. Chen, Y. H. Chen, and J. Lam, "Design of a one-motor tree-climbing robot," in *2015 IEEE International Conference on Information and Automation*, 2015, pp. 26–31.

APPENDIX A CONTRIBUTION OF EACH TEAM MEMBERS

TABLE VI
CONTRIBUTION OF EACH TEAM MEMBERS

			
Ruizhi Deng Mechanical design, manufacturing, assembly	Hengyi Zhou Electronic control programming, assembly	Dachuan Liu Programming tech communication, assembly	Letao Zhang Tech communication assembly

APPENDIX B LITERATURE REVIEW TABLE

See Table VII.

TABLE VII
LITERATURE REVIEW TABLE

No.	Method	Major findings	Unresolved issues	Refs.
L1	Gripper-based climbing mechanism	Grippers can increase the stability of climbing	Low climbing speed due to cycle mechanism	[3]–[6], [16], [17]
L2	Multi-vehicle system	Connecting several vehicles together is easy to achieve	Poor stability due to speed difference of different parts	[7], [8]
L3	Enclosing frame with wheels	The wheeled frame can ensure high speed and relatively good stability	May not be able to be applied to columns of different diameters	[9]–[12]

APPENDIX C GANTT CHART

APPENDIX D BUDGET TABLE

See Table VIII.

APPENDIX E RASPBERRY PI MOTOR CONTROL PYTHON CODE

```
import RPi.GPIO as GPIO
import time
import VL53L0X # Import VL53L0X library
import threading
import sys
import tty
import termios
from select import select

# --- GPIO pin definitions ---
PWM_FREQUENCY = 100

MOTOR_IN1 = 17 # BCM
MOTOR_IN2 = 22 # BCM
MOTOR_ENA_W = 10 # BCM
MOTOR_ENA_L = 26 # BCM
```

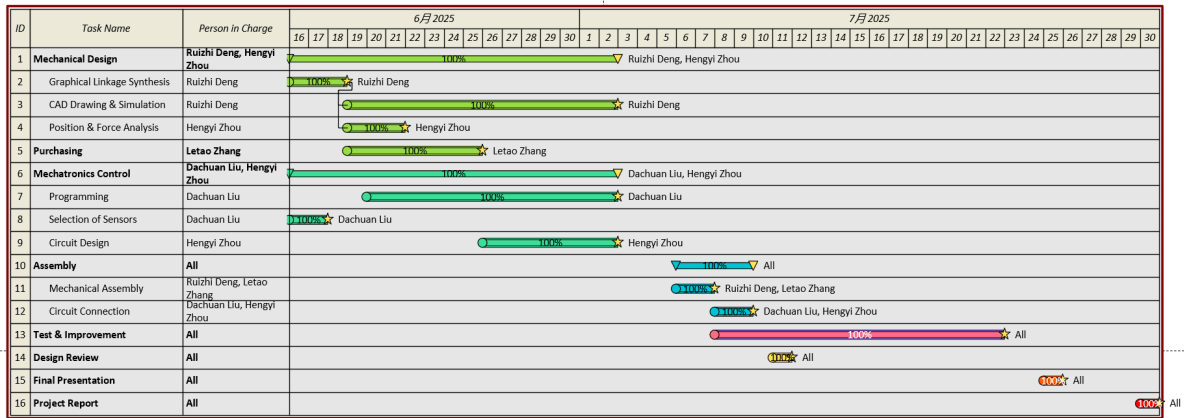


Fig. 24. Gantt chart

TABLE VIII
BILL OF MATERIALS

No.	Description	Specification	Unit	Qty	Unit Cost (¥)	Total Cost (¥)	Source	Notes
1	Lithium Battery Pack	BD18650-35-12	PC	2	55.80	111.60	Taobao	12 V, 3400 mAh
2	Laser Distance Sensor	TOF200F	PC	1	37.52	37.52	Taobao	304 Stainless
3	Silicone Rubber Wheels & Coupling	—	Set	8	34.00	272.00	Taobao	Tire diameter 68mm
4	Raspberry Pi	Raspberry Pi 4 Model B	PC	1	—	—	JI	Provided
5	Motor Driver Module	L298N	PC	2	—	—	JI	Provided
6	PC Transparent Square Tube	—	PC	6	21.82	130.92	Taobao	10 mm × 15 mm
7	Motor	JGY-370	PC	4	21.58	86.32	Taobao	—
8	Laser Distance Sensors	TOF200F+TOF400F	Set	1	84.33	84.33	Taobao	—
9	Laser Distance Sensor	YB-MVV18 VER1.0	PC	2	48.00	96.00	Taobao	—
10	Carbon Tube	—	PC	1	46.81	46.81	Taobao	Φ 12 mm, length 1.5 m
11	Aluminium Alloy Tube	—	PC	2	12.75	25.50	Taobao	Φ 15 mm, length 1 m
12	Twist Drill Bit	—	Set	1	20.90	20.90	—	—
13	Quick-release Clamp	—	PC	4	1.80	7.20	Taobao	—
14	Switch	—	PC	2	4.63	9.26	Taobao	—
15	Wires	—	PC	1	20.00	20.00	Taobao	—

```

# --- Global variables and state management ---
MOTOR_W_SPEED_UP = 90
MOTOR_W_SPEED_DOWN = 50
MOTOR_L_SPEED_UP = 90
MOTOR_L_SPEED_DOWN = 52

TARGET_HEIGHT_MIN_MM = 1090
TARGET_HEIGHT_MAX_MM = 1150
MIN_HEIGHT_MM = 110

CLIMBING = 1
DESCENDING = 2
STOPPED = 0
PAUSED = 3

motor_current_state = STOPPED
paused_state = STOPPED
wait_timer = 0
is_interrupted = False

# --- GPIO setup ---
GPIO.setmode(GPIO.BCM)
GPIO.setup(MOTOR_IN1, GPIO.OUT)
GPIO.setup(MOTOR_IN2, GPIO.OUT)
GPIO.setup(MOTOR_ENA_W, GPIO.OUT)
GPIO.setup(MOTOR_ENA_L, GPIO.OUT)

```

```

# --- PWM setup ---
pwm_motor_w = GPIO.PWM(MOTOR_ENA_W, PWM_FREQUENCY)
pwm_motor_l = GPIO.PWM(MOTOR_ENA_L, PWM_FREQUENCY)

pwm_motor_w.start(0)
pwm_motor_l.start(0)

# --- VL53L0X setup ---
tof = None
sensor_timing_budget = 200000 # microseconds

try:
    print("Initializing VL53L0X sensor...")
    tof = VL53L0X.VL53L0X(i2c_bus=1, i2c_address=0x29)
    tof.open()
    tof.start_ranging(VL53L0X.VL53L0xAccuracyMode.BEST)
    sensor_timing_budget = tof.get_timing()
    if sensor_timing_budget < 20000:
        sensor_timing_budget = 20000
    print(f"Sensor timing budget: {sensor_timing_budget/1000} ms")
    print("VL53L0X sensor initialized.")
except Exception as e:
    print(f"Error initializing VL53L0X sensor: {e}")
    print("Please ensure I2C is enabled and VL53L0X library is installed correctly.")
    tof = None

# --- Motor control functions ---
def forward_motor(speed_percent_W, speed_percent_L):
    global motor_current_state
    GPIO.output(MOTOR_IN1, GPIO.HIGH)
    GPIO.output(MOTOR_IN2, GPIO.LOW)
    pwm_motor_w.ChangeDutyCycle(speed_percent_W)
    pwm_motor_l.ChangeDutyCycle(speed_percent_L)
    motor_current_state = CLIMBING
    print(f"White Motor: Forward (Climbing) at {speed_percent_W}% speed.")
    print(f"Green Motor: Forward (Climbing) at {speed_percent_L}% speed.")

def backward_motor(speed_percent_W, speed_percent_L):
    global motor_current_state
    GPIO.output(MOTOR_IN1, GPIO.LOW)
    GPIO.output(MOTOR_IN2, GPIO.HIGH)
    pwm_motor_w.ChangeDutyCycle(speed_percent_W)
    pwm_motor_l.ChangeDutyCycle(speed_percent_L)
    motor_current_state = DESCENDING
    print(f"White Motor: Backward (Descending) at {speed_percent_W}% speed.")
    print(f"Green Motor: Backward (Descending) at {speed_percent_L}% speed.")

def stop_motor():
    global motor_current_state
    GPIO.output(MOTOR_IN1, GPIO.LOW)
    GPIO.output(MOTOR_IN2, GPIO.LOW)
    pwm_motor_w.ChangeDutyCycle(0)
    pwm_motor_l.ChangeDutyCycle(0)
    motor_current_state = STOPPED
    print("Motor: Stopped.")

# --- Keyboard input handling ---
def is_key_pressed():
    return select([sys.stdin], [], [], 0) == ([sys.stdin], [], [])

def get_key():
    old_settings = termios.tcgetattr(sys.stdin)
    try:
        tty.setcbreak(sys.stdin.fileno())
        if is_key_pressed():
            return sys.stdin.read(1)
    finally:
        termios.tcsetattr(sys.stdin, termios.TCSADRAIN, old_settings)
    return None

def reverse_direction(apply_immediately=True):
    global motor_current_state, paused_state
    new_state = None
    if motor_current_state == CLIMBING or paused_state == CLIMBING:
        print("Reversing direction: Climbing -> Descending")
        if apply_immediately:
            backward_motor(MOTOR_W_SPEED_DOWN, MOTOR_L_SPEED_DOWN)

```



```

    new_state = DESCENDING
elif motor_current_state == DESCENDING or paused_state == DESCENDING:
    print("Reversing direction: Descending -> Climbing")
    if apply_immediately:
        forward_motor(MOTOR_W_SPEED_UP, MOTOR_L_SPEED_UP)
    new_state = CLIMBING
else:
    new_state = motor_current_state
return new_state

print("Program started. Auto climbing will begin shortly...")
print("Press SPACE to pause/resume, 'r' to reverse direction")
stop_motor()

try:
    old_settings = termios.tcgetattr(sys.stdin)
    tty.setcbreak(sys.stdin.fileno())
    print("Starting auto climbing...")
    forward_motor(MOTOR_W_SPEED_UP, MOTOR_L_SPEED_UP)

while True:
    distance = -1
    if tof and not is_interrupted:
        try:
            distance = tof.get_distance()
            print(f"Current distance: {distance} mm")
        except Exception as e:
            print(f"Error reading distance: {e}")
            distance = -1

    if is_key_pressed():
        key = sys.stdin.read(1)
        if key == ' ':
            if not is_interrupted:
                paused_state = motor_current_state
                stop_motor()
                is_interrupted = True
                print("System PAUSED. Press SPACE to resume, 'r' to reverse direction.")
            else:
                is_interrupted = False
                if paused_state == CLIMBING:
                    forward_motor(MOTOR_W_SPEED_UP, MOTOR_L_SPEED_UP)
                    print("Resuming CLIMBING.")
                elif paused_state == DESCENDING:
                    backward_motor(MOTOR_W_SPEED_DOWN, MOTOR_L_SPEED_DOWN)
                    print("Resuming DESCENDING.")
                else:
                    print("Resuming previous state (was stopped).")
        elif key == 'r':
            if is_interrupted:
                paused_state = reverse_direction(apply_immediately=False)
                stop_motor()
                print(f"Direction reversed while paused. Will {'climb' if paused_state == CLIMBING else 'descend'} when resumed.")
            elif wait_timer > 0:
                print("Skipping wait time, starting descent...")
                wait_timer = 0
                backward_motor(MOTOR_W_SPEED_DOWN, MOTOR_L_SPEED_DOWN)
            else:
                print('Please pause the system first to reverse direction.')

    if not is_interrupted:
        if motor_current_state == CLIMBING and distance != -1:
            if TARGET_HEIGHT_MIN_MM <= distance <= TARGET_HEIGHT_MAX_MM:
                print(f"Reached target height: {distance} mm")
                stop_motor()
                wait_timer = 60
                print(f"Stopping for {wait_timer} seconds before descent...")
        elif motor_current_state == DESCENDING and distance != -1:
            if 20 <= distance <= MIN_HEIGHT_MM:
                print(f"Reached minimum height: {distance} mm <= {MIN_HEIGHT_MM} mm")
                stop_motor()
                print("Reached minimum height. Program finished.")
                print("Exiting program...")
                time.sleep(1)
                break
            if wait_timer > 0:

```

```

        print(f"Waiting: {wait_timer} seconds remaining...")
        wait_timer -= 1
        if wait_timer == 0:
            print("Wait time completed. Starting descent...")
            backward_motor(MOTOR_W_SPEED_DOWN, MOTOR_L_SPEED_DOWN)
            time.sleep(1)
            continue

        time.sleep(sensor_timing_budget / 1000000.0)

except KeyboardInterrupt:
    print("\nProgram terminated by user.")

finally:
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, old_settings)
    if pwm_motor_w:
        pwm_motor_w.stop()
    if pwm_motor_l:
        pwm_motor_l.stop()
    if tof:
        tof.stop_ranging()
        tof.close()
        print("VL53L0X sensor stopped and closed.")
    GPIO.cleanup()
    print("GPIO cleaned up. Program exited.")

```

A. Sensor Evaluation Chart

TABLE IX
COMPARISON OF DISTANCE SENSORS

Feature	HC-SR04	TOF200F	YB-MVV18 VER1.0
Sensing Principle	Ultrasonic	TOF (Laser)	TOF (Laser)
Measurement Range	2–400 cm	3–200 cm	1.5–200 cm
Accuracy	Low (degrades with range)	High (unstable outputs)	High ($\pm 3\%$)
Noise/Outliers	Moderate	Frequent extreme values	Minimal
Output Stability	Variable	Unstable	Stable
Communication	Digital (Pulse Width)	UART/I ² C	I ² C
Integration Ease	High	Medium	High
Size	Medium	Compact	Compact
Modes Available	Single	Fixed	GOOD/BETTER/BEST